

3 ANALYSIS

Date

Systems analysis is the part of the systems development life cycle in which we determine how the current information system functions and assess what users would like to see in a new system.

Analysis has two subphases

→ Requirements determination

→ Requirements structuring

Once the management has granted permission to pursue development of a new system and a project is initiated and planned, we begin determining what the new system should do.

Determining System Requirements

Collection of information is the most important thing for system analysis. System analysts must collect the information about the current system and how users would like to improve their performance with new information system. Accurately understanding the user's requirements will help the system developing team deliver a proper system to end users in limited time and limited budget.

Characteristics of a good systems analyst:

- * Impertinence: Ask questions about everything that exists and also about what may exist in the future.
- * Impartiality: Find the best solution to a business problem or opportunity - consider issues raised by all parties.
- * Relax constraints: Eliminate unfeasibility, assume that anything is possible
- * Attention to details: Everything must fit together so that the system works properly.
- * Reframing: Every system is different and needs a new creative approach.

Deliverables: The primary deliverables from requirements determination are various forms of information gathered during the determination process:

- Information collected from conversations with or observations of users → interview transcripts, notes from observation etc.
- Existing written information → business mission or strategy, forms, reports, training manuals, flow charts and documentation of existing system etc.
- Computer-based information → results from JRP sessions, CASE repository contents etc.

Traditional Methods for Determining Requirements

1) Interviewing and Listening: Interviewing is one of the primary ways analysts gather information about an information systems project. It involves gathering information by talking to the people who are directly or indirectly involved in the different parts of the organizations affected by the possible system changes.

Guidelines for Effective Interviewing

1) Plan the Interview

→ Prepare interviewee: appointment, priming questions.

→ Prepare checklist, agenda and questions

2) Listen carefully and take notes (record if permitted)

3) Review notes within 48 hours of interview.

4) Be neutral

5) Seek diverse views.

Types of Questions

- i) Open-ended questions : Questions in interviews that have no prespecified answers.
- ii) closed-ended questions : Questions in interviews that ask those responding to choose from among a set of specified responses.

Analysts mix and sequence of open ended and close ended questions to use. Open-ended questions are usually used to probe for information for which we cannot anticipate all possible responses or for which we do not know the precise question to ask. Close-ended questions work well when the major answers to questions are well known. Major disadvantage of this method for determining requirements is it requires huge effort and time if the number of interviewee is high.

2) Group Interviewing : Group interviewing consists of taking interview of several key people at once. Group interviewing can be performed by one or more analysts. In the case of multiple interviewers, one analyst may ask questions while another takes notes, or different analysts might concentrate on different kinds of information. The advantages of group interviewing are : more effective use of time than a series of interviews with individuals and it allows the interviewee to hear the opinions of other key people and gives them opportunity to agree or disagree with their peers. Synergies also often occur. The primary disadvantage is the difficulty in scheduling the group interview.

3) Directly Observing Users: Interviewing involves getting people to recall and convey information they have about organizational processes and the information systems that support them. People are not always reliable and say what they think is the truth. People often do not have a completely accurate appreciation of what they do or how they do it. Because people cannot always be trusted to interpret and report their own actions reliably, we can supplement what people tell us by watching what they do in work situations.

The intent behind direct observation is to obtain more firsthand and objective measures of employee interaction with information systems. In some cases, behavioral measures will be a more accurate reflection of reality than what employees themselves believe.

4) Analyzing Procedures and Other Documents:

Both interviewing and observing have some sort of limitations. Methods for determining system requirements can be enhanced by analyzing procedures and other documents of an organization.

By examining existing system and organizational documentation, system analyst can find out details about current system and the organization. In documents analyst can find information such as problems with existing system, reasons why current systems are designed, titles and names of key individuals who have an interest in relevant existing systems and many more.

Contemporary Methods for Determining System Requirements :

Even though interviews, observation and document analysis are called traditional methods for determining a system's requirement, all of these methods are still very much used by analysts to collect important information. Today, however, there are additional techniques to collect information about the current system, organizational area requesting the new system and what the new system should be like. These techniques are called contemporary methods for Determining System's requirements and can support effective information collection and structuring while reducing the amount of time required for analysis.

Joint Application Design (JAD)

JAD is a team based approach for defining the requirements for ~~new~~ new or modified systems. The main idea behind JAD is to bring together the key users, managers and system analysts involved in the analysis of a current system. The primary purpose of JAD is to collect system requirements simultaneously from the key people involved in the system. The result is an intense and structured, but highly effective process. Having all the key people together in one place at one time allows analyst to see where there are areas of agreement and where there are conflicts.

JAD sessions are usually conducted at a location other than the place where the people involved normally work keeping participants away from as many distractions as possible. A JAD may last anywhere

from four hours to an entire week and may consist of several sessions. The typical participants in a JAD are :

- 1) JAD session leader
- 2) Users
- 3) Managers
- 4) Sponsor
- 5) Systems analysts
- 6) Scribe
- 7) IS staff

Prototyping :

Prototyping is an iterative process involving analysts and users whereby a rudimentary version of an information system is built and rebuilt according to user feedback. Prototyping can augment the requirements determination process by allowing system analysts quickly show users the basic requirement into a working version of the desired system. Typically, seeing verbal descriptions of requirements converted into a physical system will prompt the user to modify existing requirements and generate new ones.

As the prototype changes through each iteration, more and more of the design specifications for the system are captured in the prototype. The prototype can then serve as the basis for the production system itself, this process is called evolutionary prototyping.

Alternatively, the prototype can serve only as a model which is then used as a reference for the construction of the actual system. The prototype is discarded after it has been used. This process

is called throwaway prototyping.

Prototyping is most useful for requirements determination when:

- User requirements are not clear or well understood.
- One or a few users and other stakeholders are involved with the system.
- Possible designs are complex and require concrete form to fully evaluate.
- Communication problems have existed in the past between users and analysts and both parties want to be sure that system requirements are as specific as possible.
- Tools (such as form and report generators) and data are readily available to rapidly build working systems.

Drawbacks as a tool for requirements determination:

- A tendency to avoid creating formal documentation of system requirements, which can then make the system more difficult to develop into fully working system.
- Prototype can become idiosyncratic to the initial user and difficult to diffuse or adapt to other potential users.
- Prototypes are often built as stand-alone systems, thus ignoring issues of sharing data and interactions with other existing systems.
- Checks in the SDLC are bypassed so that some more subtle, but still important, system requirements might be forgotten (e.g., security, some data-entry controls, or standardization of data across systems).

Radical Methods for Determining System Requirements

1) Business Process Reengineering: The overall process by which current methods are replaced with radically new methods is referred to as business process reengineering (BPR). Reorganize complete flow of data in major sections of organization, Eliminate unnecessary steps, combine steps, and Become more responsive to future change are its main goals. This method search for implementation of radical change in business process to achieve breakthrough improvements in products and services.

Identification of process to reengineer:

The first step in any BPR effort is to understand what processes need to change, what are the key business processes for the organization. Key business processes are the structured set of measurable activities designed to produce a specific output for a particular customer or market. Key business processes are customer focused. After identifying key business processes, the next step is to identify specific activities that can be radically improved through reengineering.

Michael Hammer and James Champy, who coined the term BPR, suggest systems analysts ask three questions to identify activities for radical change:

1. How important is the activity to delivering an outcome?
2. How feasible is changing the activity?
3. How dysfunctional is the activity?

The answers to these questions provide

guidance for selecting which activities to change. Those activities deemed important, changeable, yet dysfunctional, are primary candidates. To identify dysfunctional activities, they suggest to look for activities where there are excessive information ~~exchange~~ exchanges between individuals, where information is redundantly recorded or needs to be rekeyed, where there are excessive inventory buffers or inspections and ~~there~~ where there is a lot of rework or complexity.

Disruptive Technologies: Technologies that enable, breaking long-held business rules that discourage organizations from making radical business changes are called disruptive technologies. In this method problems are first identified and then solutions are formulated.

For example, Information can appear in only one place at a time is a long-held organizational rule. However distributed databases and pervasive wireless networking have "disrupted" this belief.

* Advantages of JAD:

- JAD reduces costs and time for project development.
- JAD encourages the team to push each other to work faster and deliver on time.
- JAD allows us to produce better, error-free, software.
- JAD lowers the risks
- JAD improves system quality.

Disadvantages

- Different opinions within the team make it difficult to align goals and maintain focus.
- Depending on the size of project, JAD may require a significant time commitment.

Structuring System Process Requirements:

Process Modeling: Process modeling involves graphically representing the processes, or actions, that capture, manipulate, store and distribute data between the system and its environment and among components within the system. A common form of a process model is data-flow diagram (DFD).

Modeling a System's Process for structured Analysis

As part of structuring, informations collected during system's requirements determinations are organized into a meaningful representation of the information system that exists and of the requirements desired in a replacement system. In addition to modeling the processing elements of an information system and transformation of data in the system, structure of data within the system is also modeled. Analysts use both process and data models to establish the specification of an information system. With a supporting tool, such as a CASE tool, process and data models can also provide the basis for the automatic generation of an information system.

Deliverables and Outcomes: In structured analysis, the primary deliverables from process modeling are set of coherent, inter-related data-flow diagrams. Following are the deliverables for process modeling:

- Context DFD
- DFDs of current physical system.
- DFDs of new logical system.
- Through description of each DFD component.

Data Flow Diagrams

Graphical representation of a system's data and how the processes transform the data is known as Data Flow diagram (DFD). Unlike flowcharts, DFD's do not give detailed descriptions of modules but graphically describe a system's data and how the data interact with the system.

Structure of DFD allows starting from a broad overview and expands it to a hierarchy of detailed diagrams. DFD has often been used due to the following reasons:

- Logical information flow of the system.
- Determination of physical system construction requirements.
- Simplicity of notation
- Establishment of manual and automated system's requirements.

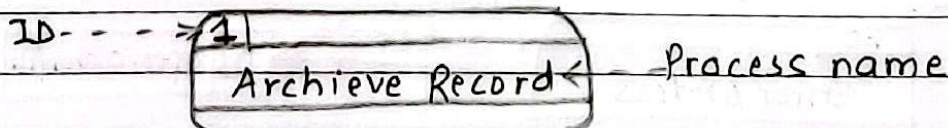
Symbols used to represent data-Flow diagram (components of DFD)

1) Process: A process receives input data and produces output. Every process has a name that identifies the function it performs, The name consists of a verb, followed by a singular noun.

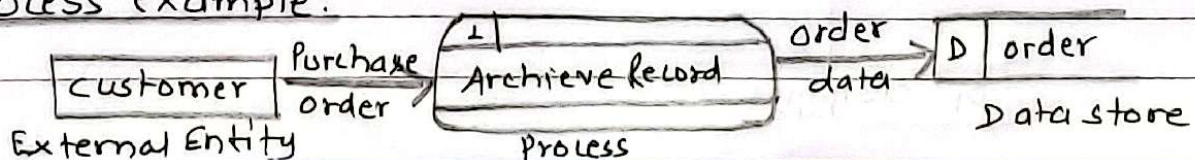
Examples: Apply payment, Verify order etc.

Notation: A rounded rectangle represents a process.

Process are given IDs for easy referencing



Process Example:



2> Data Flow: A data-flow is a path for data to move from one part of the information system to another. A data-flow may represent a single data element such as customer ID or it can represent a set of data element (or a data structure).

Notation:

→ Straight lines with incoming arrows are input data flows.

→ Straight lines with outgoing arrows are output data flows.

3> Data Store: A data-store is used in DFD to represent a situation when the system must retain data because one or more processes need to use the stored data in a later time. A data-store must be connected to a process with a data-flow. Each data store must have at least one input data flow and at least one output data-flow.

Notation:

→ Data can be written into data store, representing by outgoing arrows

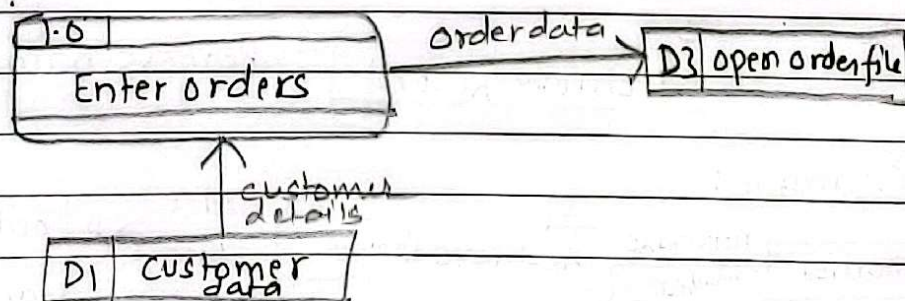
→ Data can be read from data store, representing by an incoming arrow.

Label →  Inventory ← data store name

OR

Label →  Inventory ← data store name.

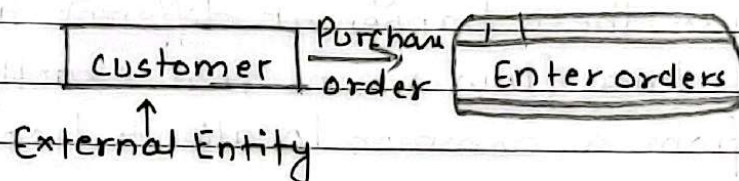
Example:



4) External Entity: An external entity is a person, department, outside organization, or other information system that provides data to the system or receives output from the system. External entities are also called terminators (source / sink) because they are data origins (source) or final destinations (sink). An external entity must be connected to a process through a data flow.

Notation: A rectangle represents an external entity.

Examples



Rules of Data Flow:

Process:

- No process can have only outputs. It is making data from nothing (a miracle). If an object has only outputs, then it must be a source.
- No process can have only inputs (a black hole). If an object has only inputs, then it must be a sink.
- A process has a verb phrase label.

Data store:

- Data cannot move directly from one data store to another data store. Data must be moved by a process.
- Data cannot move directly from an outside source to a data store. Data must be moved by a process that receives data from the source and places the data into the data store.
- Data cannot move directly to an outside sink from a data store. Data must be moved by a process.
- A data store has a noun phrase label.

Source / Sink:

- Data cannot move directly from a source to a sink.

It must be moved by a process if the data are of any concern to our system. Otherwise, the data flow is not shown on the DFD.

I. A Source/Sink has a noun-phrase label.

Data Flow:

J. A data flow has only one direction of flow between symbols. It may flow in both directions between a process and a data store to show a read before an update. The latter is usually indicated by two separate arrows because these happen at different times.

K. A fork in a data flow means that exactly the same data goes from a common location to two or more different processes, data stores, or sources/sinks. (this usually indicates different copies of the same data going to different locations)

L. A join in a data flow means that exactly the same data come from any of two or more different processes, data stores, or sources/sinks to a common location.

M. A data flow cannot go directly back to the same process it leaves. There must be at least one other process that handles the data flow, produces some other data flow, and returns the original data flow to the beginning process.

N. A data flow to a data store means update (delete or change).

O. A data flow from a data store means retrieve or use.

P. A data flow has a noun phrase label. More than one data flow noun phrase can appear on a single arrow as long as all of the flows on the same arrow move together as one package.

Incorrect and correct ways to draw DFDs.

Rule

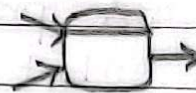
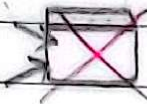
Incorrect

Correct

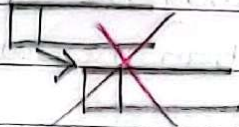
A.



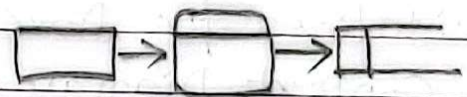
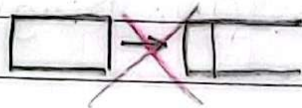
B.



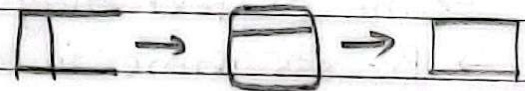
D.



E.



F.



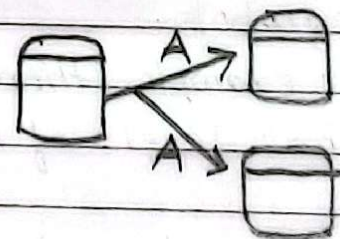
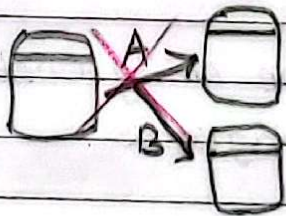
H.



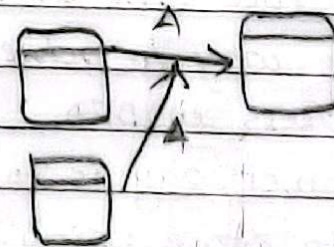
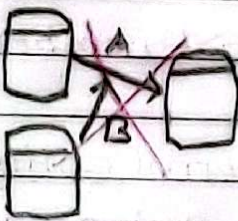
J.



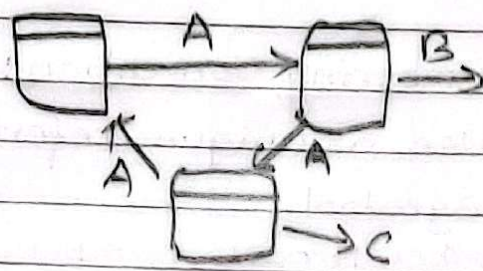
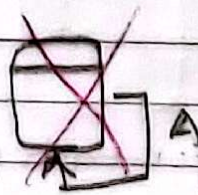
K.



L.



M.



frequently-made mistakes in DFD.

Black holes - A processing step may have input flows but no output flows.

Miracles - A processing step may have output flows but no input flows.

Grey holes - A processing step may have outputs that are greater than the sum of its input.

Guidelines for Developing Data-flow Diagram

1. Context Diagram - level 0:

- The context diagram must fit in one page.
- Process name in context diagram should be the name of the information system. e.g. Grading system.
- The context level diagram gets the number 0.

2. Unique Name for levels:

- Unique names should be used for each set of symbols.
- for example, there can be only one entity Customer in all levels of the data-flow diagrams.

3. No Cross Line in DFD:

- One way to achieve this is to restrict number of process in DFD.
- Another way is to duplicate an external entity or data store using a special notation such as asterisk.

4. Numbering Convention:

- Use a unique reference number for each process symbol.
- other process numbers are in the hierarchy of:
 - (1, 2, 3, ...)
 - (1.1, 1.2, ..., 2.1, 2.2, ...)

→ (1.1.1, 1.1.2, ...)

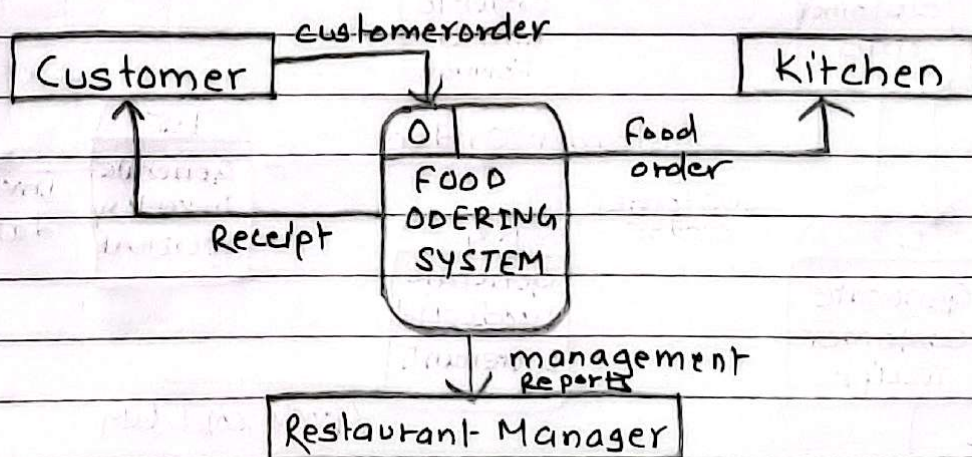
Context Level Diagram :

A context diagram gives an overview and it is the highest level in a data flow diagram, containing only one process representing the entire system.

- All external entities are shown on the context diagram as well as major data flow to and from them.
- The diagram does not contain any data storage.
- The single process in the context-level diagram, representing the entire system, can be exploded to include the major processes of the system in the next level diagram, which is termed as diagram 0.

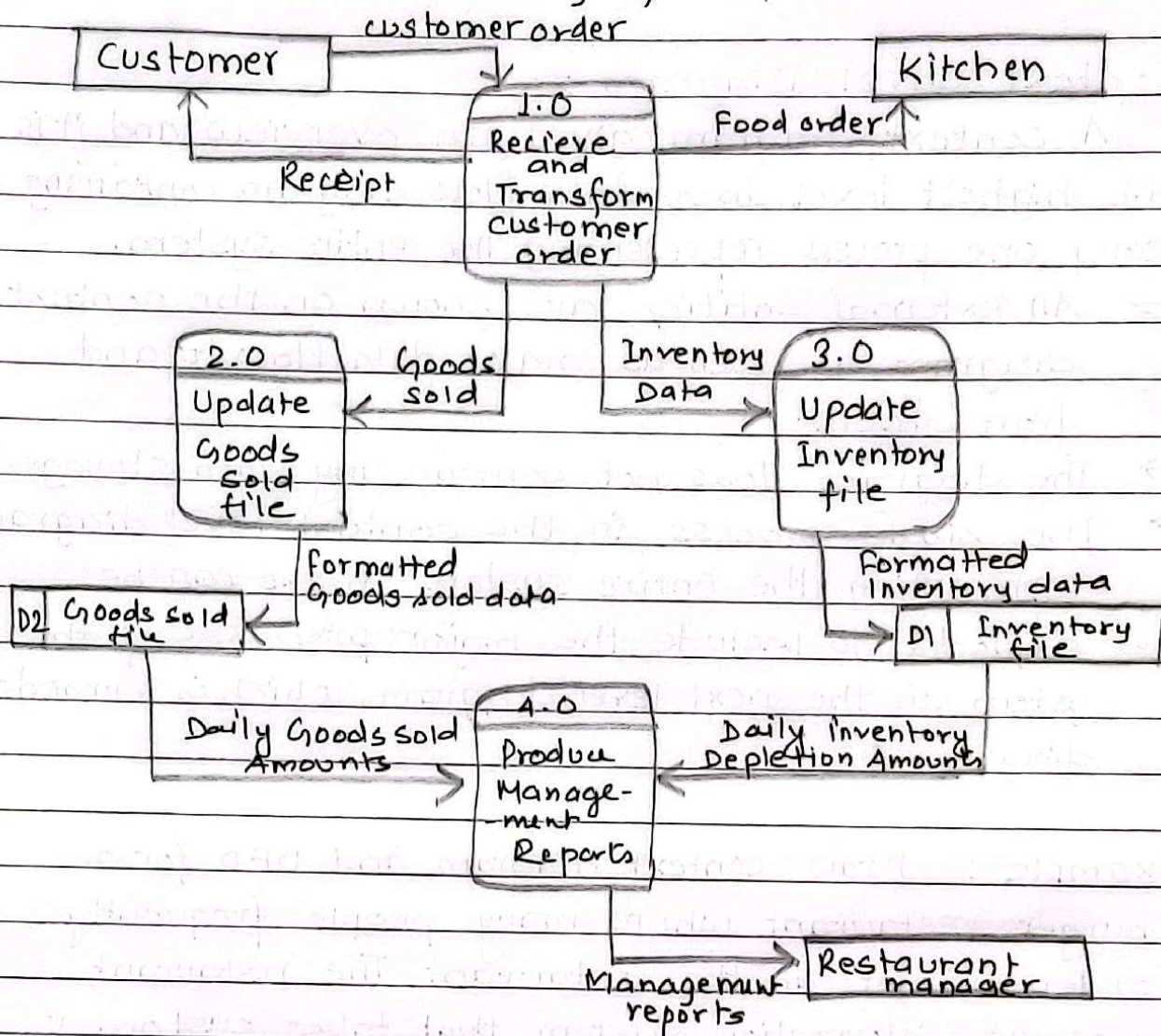
Example : Draw context diagram and DFD for a burger restaurant where many people frequently order burger at the restaurant. The restaurant uses an information system that takes customer orders, sends the orders to the kitchen, monitors goods sold and inventory, and generates reports for management.

Context Diagram :

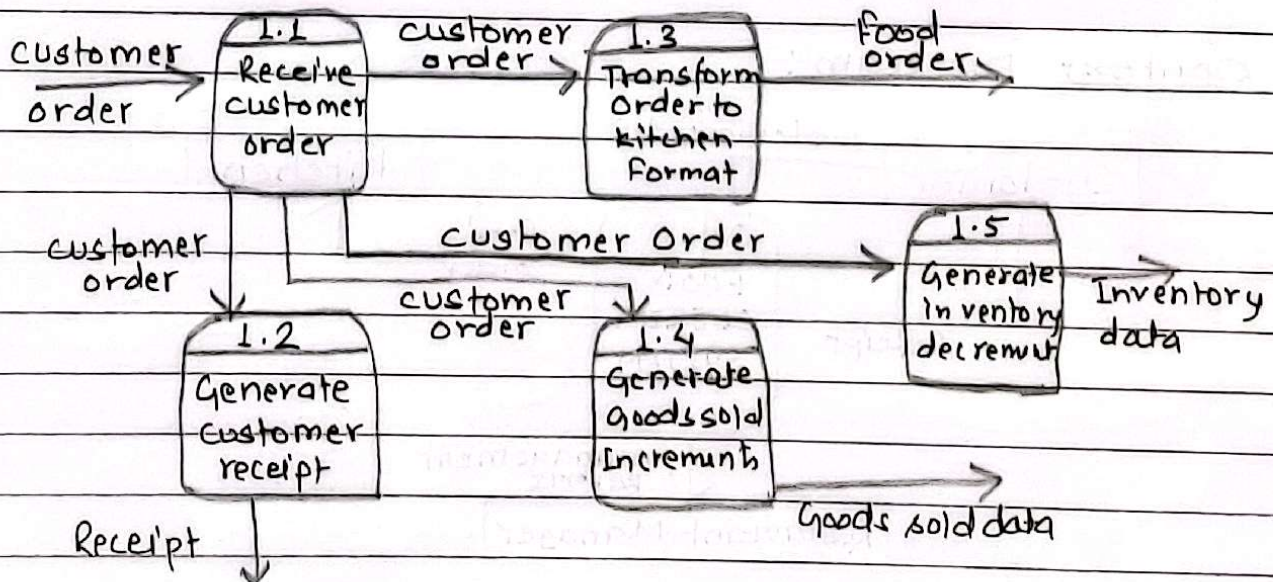


Date

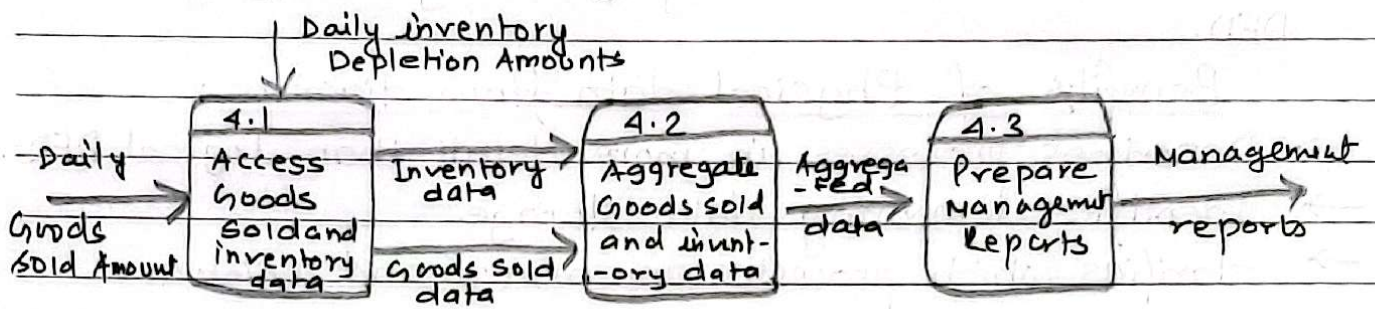
Level-0 DFD (containing four major processes)



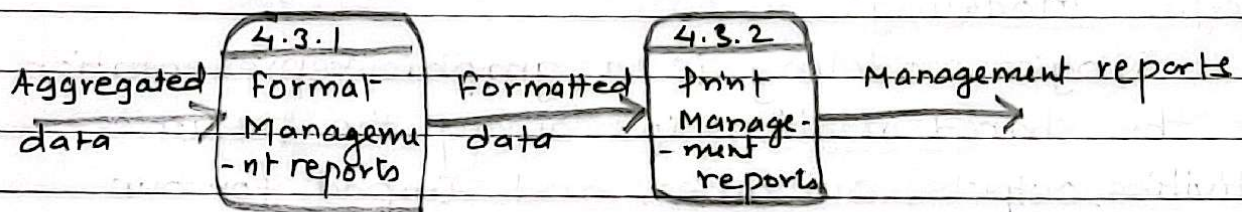
Level-1 diagram showing decomposition of process 1.0 from level-0 diagram:



Level-1 diagram showing decomposition of process 4.0 from level-0 diagram:



Level-2 diagram showing decomposition of process 4.3



Logical vs Physical Data Flow Diagrams:

A logical data flow diagram focuses on the business and how the business operates. It is not concerned with how the system will be constructed. We can ignore implementation specifics such as, computer configuration, data storage technology etc.

A physical data flow diagram shows how the system will be implemented, including the hardware, software, files, and people in the system. It is developed such that the processes described in the logical data flow diagrams are implemented correctly to achieve the goal of the business.

Benefits of Logical data flow diagram:

- It is based on business events and independent of particular technology.
- It allows analyst to understand the business being ~~started~~ ^{studied}.
- Systems Implemented based on Logical DFD will be easier to

maintain

- Physical DFD can be easily formed by modifying a logical DFD.

Benefits of Physical data flow diagram:

- Describes processes in more detail than logical DFDs.
- Identifies temporary data storage.
- Clarifies which processes are manual and which are automated.
- Adds controls to ensure the processes are done properly.

Logic Modeling:

Logic modeling is a graphic representation for the shared relationships among the resources, activities, outputs, outcomes, and impact for our program.

Data flow diagrams do not show the logic inside the processes. Logic modeling involves representing internal structure and functionality of processes depicted on a DFD. Processes must be clearly described before translating them into programming language. Logic modeling can also be used to show when processes on a DFD occur.

A logic model has four components:

1. Needs: Needs are about the problem and why it's important to address it. What issue are we trying to address?
2. Inputs: These are the things that contribute to addressing the need. What resources are we investing?
3. Activities: These describe the things that the inputs allow to happen. What are we doing with these resources?
4. Outcomes: Outcomes are usually expressed in

terms of measures of success. What difference are we hoping to make.

Modeling Logic with Decision Tables:

Decision tables are a concise visual representation for specifying which actions to perform depending on given conditions. They are algorithms whose output is a set of actions. It is an excellent tool to use in both testing and requirement management.

Decision table is a matrix representation of the logic of a decision, which specifies the possible conditions for the decision and the resulting actions. It consists of three parts: Condition stubs, Action stubs and Rules.

Condition stubs: The part of a decision table that lists the conditions relevant to the decision.

Action stubs: The part of a decision table that lists the actions that result for a given set of conditions.

Rules: The part that specifies which actions are to be followed for a given set of conditions.

Example: Decision table for ATM withdrawal process

	Conditions / courses of Action	Rules		
		1	2	3
Condition Stubs	Withdrawal amount $\leq$ balance	T	F	F
	Credit Granted	-	T	F
Action Stubs	Withdrawal granted	T	T	F

In a decision table, conditions are usually expressed as True (T) or False (F). The above table contains three different columns which are business rules, like one of the

business rule for ATM is that machine pays out if the customer has sufficient funds in their account.

Steps to create decision tables:

Step 1: Analyze the requirements and create the first column. Express conditions and resulting actions in a list.

Example,

<u>Conditions</u>
Withdrawal amount $\leq$ balance
Credit granted
<u>Actions</u>
Withdrawal granted

Step 2: Add Columns.

Number of columns needed = 2^{conditions}.

Example:

We will require 4 columns, since no. of conditions is 2.

Now, T (True) and F (False) for conditions are filled. For each row, twice as many T and F as the previous line is filled.

Example

Conditions	Withdrawal amount $\leq$ balance	T	F	T	F
	Credit granted	T	T	F	F
Actions	Withdrawal granted				

Step 3: Reduce the table

- Mark insignificant values with "-".
- delete columns that have become identical.

Example:

Conditions	Withdrawal amount $\leq$ balance	T	F	T	F
	Credit granted	-	T	-	F
Actions	Withdrawal granted				

Here, first and third columns are identical so we can delete third column.

Conditions	Withdrawal amount $\leq$ balance	T	F	F
	Credit granted	-	T	F
Actions	Withdrawal granted			

Step 4: Determine actions.

→ Enter actions for each column in the table. We ~~will~~ could find it in the requirement.

→ Name the column (the roles)

Example

	Conditions / Actions	Rules		
		1	2	3
Conditions	Withdrawal amount $\leq$ balance	T	F	F
	Credit granted	-	T	F
Actions	Withdrawal granted	T	T	F

Step 5: Write test cases

→ Write test cases based on the table. At least one test case per column gives full coverage of all business rule.

Example

* Test case for rule 1: balance = 200, requested withdrawal = 200, Expected result: withdrawal granted.

* Test case for rule 2: balance = 100, requested withdrawal = 200, ~~Expected result~~ credit granted. Expected result = withdrawal

granted.

Test case for rule 3: balance = 100, request withdrawal = 200, no credit. Expected Result: withdrawal denied.

Advantages of decision table

- They are easier to draw.
- Decision table can be easily changed according to the situation.
- A small table can replace several pages of flowchart.
- The requirements become much clearer.
- Decision tables ~~must~~ make it possible to detect combinations of conditions.

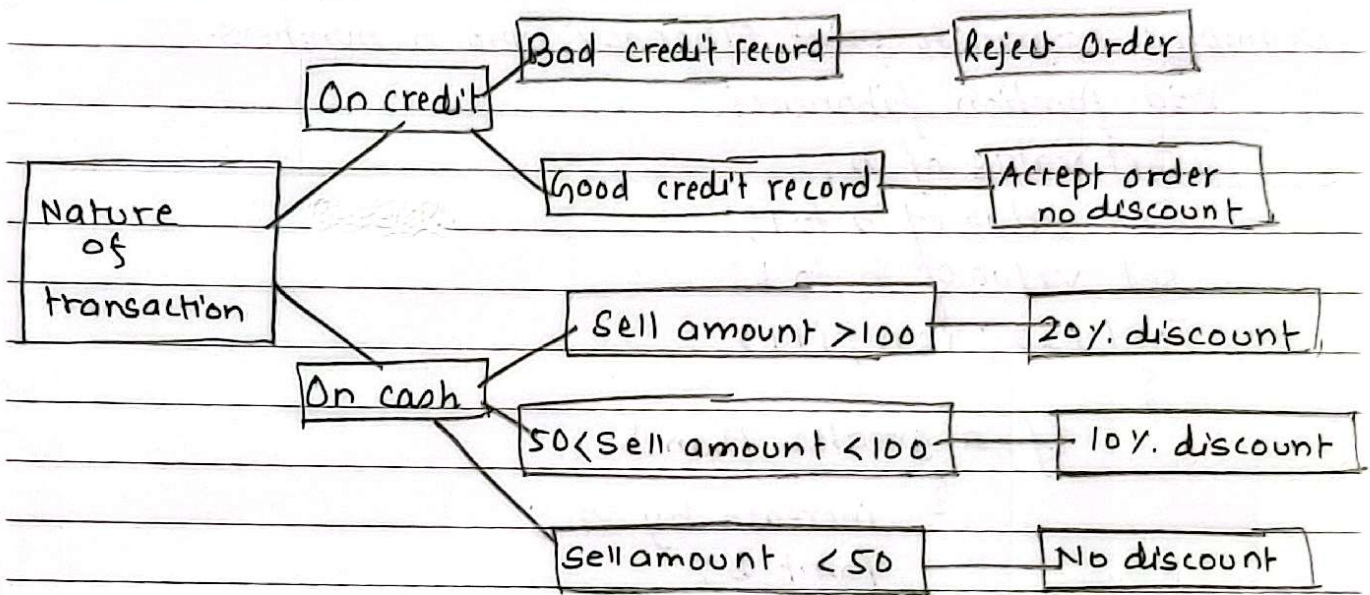
Disadvantages of decision table:

- When there are too many alternatives, decision table cannot list them all.
- It can not express complete sequence of operations to solve a problem.
- They only present partial solution.

Modeling Logic with Decision Trees:

A decision tree is a flowchart like tree structure, where each internal node denotes a test on an attribute, each branch represents an outcome of a test, and each terminal node holds a class label. Decision trees are method for defining complex relationships by describing decisions and avoiding the problems in communication. Decision trees depict the relationship of each condition and their permissible actions. A square node indicates an action and a circle indicates a condition. It forces analysts to consider the sequence of decisions and identifies the actual decision that must be made.

Example: Decision making on discount offering using decision tree.



Advantages of decision tree

- It expresses the logic of if then else in pictorial form.
- It helps the analyst to identify the actual decision to be made.
- It is used to verify logic and problems that involve a few complex decision and limited numbers of actions.
- Useful to express the decision based on nested conditions (or simply nested decision).

Disadvantages of decision tree

- There is absence of information in its format to take what other combinations of conditions to test.
- A large number of branches with many paths will confuse rather than help in analysis.

Modeling logic with Pseudo-codes:

Pseudo-code is an informal way of programming description that does not require any strict programming language syntax. It is used for creating an outline or a rough draft of a program. It has no syntax like any of the programming

language and thus can't be compiled by the computer.

Example: Program to print Fibonacci upto n numbers.

void function fibonacci

Get value of n;

Set value of a to 1;

set value of b to 1;

For($i=0$; $i \leq n$; $i++$)

{

 If a greater than b

 {

 increase by a;

 Print b;

 }

 Else if b greater than a

 {

 Increase a by b;

 Print a;

 }

}

Advantages of Pseudo-code.

- Improves readability of any approach.
- It is one of the best approach to start implementation of an algorithm.
- Acts as a bridge between the program and the algorithm or flowchart.
- Explains what exactly each line of a program should do.

Structuring System Data Requirements :

Structuring system and data requirements concentrates on the definition, structure and relationships within data. The characteristics of data captured during data modeling are crucial in the design of databases, programs, computer screens and printed reports. This information is essential in ensuring data integrity in an information system. The most common format used for data modeling is entity-relationship diagramming.

Conceptual Data Modeling

A conceptual data model is a map of concepts and their relationships used for databases. It is a representation of organizational data. Its purpose is to show as many rules about the meaning and interrelationships among data as possible.

Conceptual data modeling is typically done in parallel with other requirements analysis and structuring steps during system analysis. On larger systems a subset of project team concentrates on data modeling while other team members focus on process or logic modeling. Analysts develop a conceptual data model for the current system and then build or refine a conceptual data model that supports the scope and requirements for the proposed or enhanced system.

The process of conceptual data modeling

This process usually begins with developing a conceptual data model for the existing system. Then a new CDM, which includes all of the data requirements for the new system is built. E-R diagrams can be translated into wide variety of

technical architectures for data, such as relational, network and hierarchical.

Deliverables and Outcomes

The primary deliverables from the conceptual data modeling is the entity-relationship diagram. Another deliverable from CDM is a full set of entries about data objects to be stored in the project dictionary or repository. The repository is the mechanism to link data, process and logic models of an information system.

Gathering Information for Conceptual data Modeling

Requirements determination methods must include questions and investigations that take a data, not only a process and logic, focus. to gain the perspective on data that will be needed to develop or tailor a purchased data model.

Data modeling is typically done from a combination of perspectives. The first perspective is generally called the top-down approach. This perspective derives the business rules for a data model from an intimate understanding of the nature of the business, rather than from any specific information requirements in computer displays, reports or business forms.

Information can also be gathered for data modeling by reviewing specific business documents (reports, business forms etc) handled within the system. This perspective is called a bottom-up approach. These items will appear as data flows on DFDs and will show the data processed by the system.

Introduction to E-R Modeling

An entity-relationship data model (E-R Model) is a detailed, logical representation of the data for an organization or for a business area. The E-R model is expressed in terms of entities in the business environment, the relationships or associations among those entities, and the attributes or properties of both the entities and their relationships.

An E-R model is normally expressed as an Entity-relationship diagram (E-R diagram), which is a graphical representation of an E-R model.

Elements of E-R Diagram / ER Design Issues

1) Entity: Entity is a person, place, object, event or concept in the user environment about which the organization wishes to maintain data. Entity is a thing or object in the real world with an independent existence either with a physical existence or with conceptual existence.

For example:

- Person : Employee, Student, Patient
- Place : Store, warehouse, state
- Object : Machine, Building, Product
- Event : Sale, Registration, Renewal
- Concept : Account, Course, Workcenter

* Entity type (Entity class) : A collection of entities that share common properties or characteristics or attributes. E.g Student

* Entity instance : A single occurrence of an entity type.

* Entity set : It is same as an entity type, but defined at a particular point in time, such as students enrolled on class on the first day.

2) Attributes: The named properties or characteristics of an entity that is of interest to the organization. The particular properties that describe entity are known as attributes.

Some typical entity types and associated attributes:

STUDENT: Student ID, Student Name, Phone Number

AUTOMOBILE: Vehicle ID, Color, Weight, Horsepower

EMPLOYEE: Employee ID, Employee Name, Age, Salary

Types of attributes:

i) Atomic Vs Composite → An attribute that cannot be divided into smaller independent attribute is known as atomic attribute.

Composite attributes are those, that can be divided. They are attributes that has meaningful component parts.

ii) Single valued Vs Multi-valued → An attribute having only single value for an entity is single valued attribute.

The attribute that may take on more than one value for each entity instance are multivalued attributes.

iii) Key attribute → An attribute that uniquely identifies each instance of an entity type. i.e. An attribute that has unique value of each instance is known as key attribute.

iv) Repeating group → A set of two or more multivalued attributes that are logically related.

v) Required attribute → An attribute that must have a value for every entity instances.

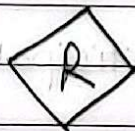
vi) Optional attribute (Null attribute): An attribute that may not have a value for every entity instances.

vii) Derived attribute: An attribute whose value can be computed from related attribute values.

Different Symbols Used in E-R diagram:

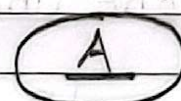
 → Entity set

 → Weak entity

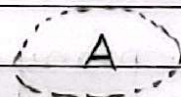
 → Relationship

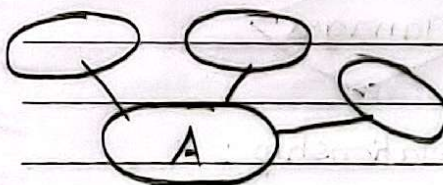
 → Identifying Relationship

 → Attribute

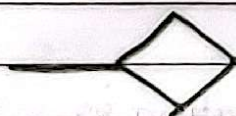
 → Key attribute

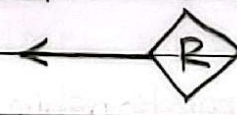
 → Multivalued attribute

 → Derived attribute

 → composite attribute

 → many-to-many relationship

 → many-to-one relationship

 → one-to-one relationship

 → Total generalization

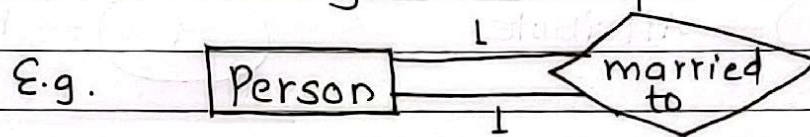
 → ISA (specialization or generalization)

3> Relationships: Relationship is an association between the instance of one or more entity types that is of interest to the organization. An association usually means that an event has occurred or that there exists some natural linkage between entity instances. Relationships are labeled with verb phrases.

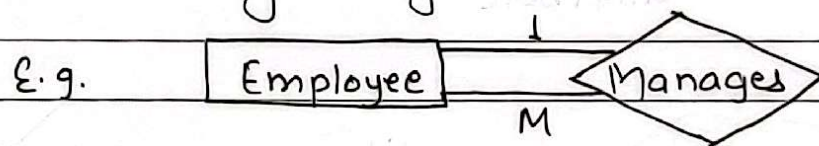
Degree of Relationship: Number of entity sets that participate in a relationship is called degree of the relationship. On the basis of degree, relationships can be divided as below:

i> Unary relationship: Only one entity set participate in a relation.

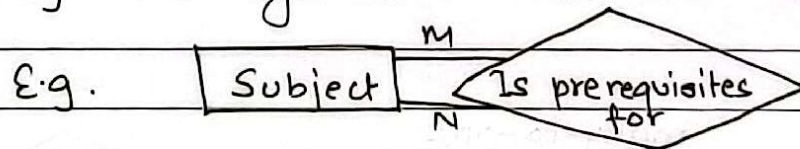
a) One-to-one unary relationship (1:1):



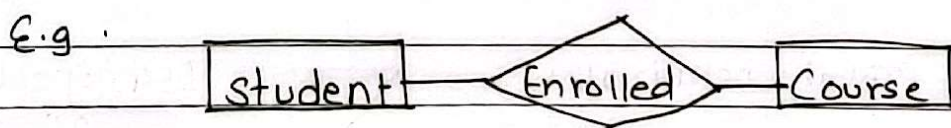
b) One-to-many unary relationship (1:M):



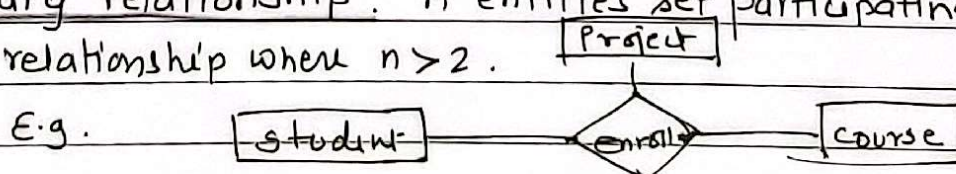
c) Many-to-many (M:N) unary relationship:



ii> Binary relationship: Two entities set participating in a relation.



iii> N-ary relationship: n entities set participating in a relationship when $n > 2$.



Cardinality: The cardinality of a relationship is the number of instances of one entity that can be associated with instances of another entity.

Suppose there are two entities type, A and B connected by relationship. The cardinality of a relationship is the number of instances of entity B that can (must) be associated with each instance of entity A.

Types

↳ One-to-one

↳ One-to-many

↳ many-to-one

↳ many-to-many

Naming Conventions:

When designing a database schema, the choice of names for entity types, attributes, relationship types, and roles is not always straight forward. One should choose names that convey, as much as possible, the meanings attached to the different constructs in schema. We choose to use singular names for entity types, rather than plural ones, because the entity type name applies to each individual ~~ent~~ instances of that entity type. In ER diagram, general convention are: entity type and relationship type name are Uppercase letters, attribute names have their initial letter capitalized and role names are lowercase letters.

Associative entity: An entity type that associate the instances of one or more entity types and contains attribute that are peculiar to the relationship between those entity instances. It is also called a gerund. It is represented by diamond inside a rectangle. i.e.

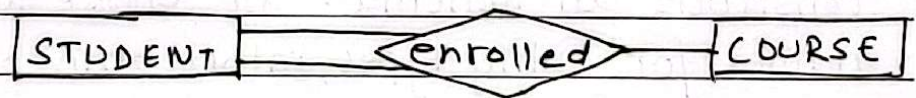


Participation constraints: Participation Constraint is applied on the entity participating in the relationship set to maintain the data integrity. Constraints on ER model that determines whether all or only some instances of entity participate in relationship is called participation constraints.

Types:

- i) Total participation: Every entity in a set must participate in the relationship. It is typically represented with a double line connecting the entity set to the relationship set.

Example,



Here STUDENT entity has Total participation i.e. each student must be enrolled to at least one course.

- ii) Partial/Optional Participation: Some entities in a set may not participate in the relationship. It is represented with a single line connecting the entity set to the relationship set.

Example:

In above example entity COURSE has partial participation. i.e. Some courses might not take part in relationship.

Real-world Scenarios: If an entity needs to be related to another entity for its existence we should use total participation. If the relationship is not essential, we should use partial participation.

Solved Questions

- 1] Construct an E-R diagram for a car-insurance company whose customers own one or more cars each. Each car has associated with it zero to any number of recorded accidents. Assume attributes of your own interest:

